

CustomerServiceTest				
CP	Método / Endpoint / Manejador	Descripción	Responsable	
CP-CS01	getCustomers()	Retorna lista de clientes cuando existen	Angi	Pasó
CP-CS02	getCustomers()	Retorna lista vacía cuando no hay clientes	Angi	Pasó
CP-CS03	getCustomerById()	Retorna cliente cuando existe	Angi	Pasó
CP-CS04	getCustomerById()	Lanza excepción cuando cliente no existe	Angi	Pasó
CP-CS05	createCustomer()	Debe crear cliente cuando las validaciones pasan	Angi	Pasó
CP-CS06	createCustomer()	Debe lanzar excepción cuando las validaciones fallan	Angi	Pasó
CP-CS07	updateCustomer()	Debe actualizar cliente cuando las validaciones pasan	Angi	Pasó
CP-CS08	updateCustomer()	Debe lanzar excepción cuando cliente no existe al actualizar	Angi	Pasó
CP-CS09	updateCustomer()	Debe lanzar excepción cuando validaciones fallan	Andrea	Pasó
CP-CS10	deleteCustomer()	Elimina cliente cuando no tiene cuentas activas	Andrea	Pasó
CP-CS11	deleteCustomer()	Lanza excepción cuando cliente tiene cuentas activas	Andrea	Pasó
CustomerControllerTest				
CP-CS12	createCustomer POST /customers	Retorna 201 created con cliente creado	Andrea	Pasó
CP-CS13	getCustomers GET /customers	Retorna 200 OK con lista de clientes	Andrea	Pasó
CP-CS14	getCustomerById GET /customers/{id}	Retorna 200 OK con cliente específico	Andrea	Pasó
CP-CS15	updateCustomer PUT /customers/{id}	Retorna 200 OK con cliente actualizado	Andrea	Pasó
CP-CS16	deleteCustomer DELETE /customers/{id}	Retorna 204 No Content	Andrea	Pasó
CP-CS17	getCustomers GET /customers	Retorna 200 OK con lista vacía	Angi	Pasó
CP-CS18	getCustomerById GET /customers/{id}	Retorna 404 Not Found lanza una excepción si el cliente no existe	Angi	Pasó
CP-CS19	createCustomer POST /customers	Retorna 400 lanza una excepción si DNI existe	Angi	Pasó
CP-CS20	updateCustomer PUT /customers/{id}	Retorna 404 Not Found lanza una excepción si el cliente no existe	Angi	Pasó
GlobalExceptionHandlerTest				
CP-CS21	MethodArgumentNotValidException	Retorna 400 con errores de validación	Andrea	Pasó
CP-CS22	DniAlreadyExistsException	Retorna 400 con mensaje apropiado	Andrea	Pasó
CP-CS23	CustomerNotFoundException	Retorna 404 con mensaje apropiado	Andrea	Pasó
AccountValidationServiceTest				
CP-CS24	validateCustomerHasNoAccounts()	Debe lanzar excepción si tiene cuenta(s) asociadas	Stephanie	Pasó
CP-CS25	validateCustomerHasNoAccounts()	Debe lanzar excepcion si existe errores del servidor	Stephanie	Pasó
CP-CS26	validateCustomerHasNoAccounts()	Debe lanzar excepción si existe errores de conexión	Stephanie	Pasó
CP-CS27	validateCustomerHasNoAccounts()	Debe lanzar excepción si el Response es null	Stephanie	Pasó

CP-CS28	validateCustomerHasNoAccounts()	Debe lanzar excepción si el Response contenga el array vacío	Stephanie	Pasó
ValidationContextTest				
CP-CS29	executeValidations()	Debe llamar a todas las estrategias en orden	Stephanie	Pasó
CP-CS30	executeValidations()	Debe para en la primera excepción	Stephanie	Pasó
CP-CS31	executeValidations()	Debe manejar las estrategias de lista vacía	Stephanie	Pasó
CP-CS32	executeValidations()	Debe manejar el customerId nulo	Stephanie	Pasó
DniValidationStrategyTest				
CP-CS33	validate()	Debe lanzar excepción cuando existe DNI para nuevo cliente	Stephanie	Pasó
CP-CS34	validate()	Debe lanzar excepción si DNI existe para otro cliente	Stephanie	Pasó
CP-CS35	validate()	Debe lanzar excepción si el DNI no existe	Stephanie	Pasó

AccountServiceTest				
CP	Método / Endpoint / Manejador	Descripción	Responsable	Pasó?
CP-AS01	getAccounts()	Debe retornar lista de cuentas cuando existen	Stephanie	Pasó
CP-AS02	getAccounts()	Debe retornar lista vacía cuando no hay cuentas	Aracely	Pasó
CP-AS03	getAccountById()	Debe retornar cuenta cuando existe	Aracely	Pasó
CP-AS04	getAccountById()	Debe lanzar excepción cuando no existe	Aracely	Pasó
CP-AS05	getAccountByCustomerId()	Debe retornar cuentas del cliente	Stephanie	Pasó
CP-AS06	getAccountByCustomerId()	Debe retornar lista vacía si no tiene cuentas	Stephanie	Pasó
CP-AS07	createAccount()	Debe crear cuenta cuando cliente existe	Aracely	Pasó
CP-AS08	createAccount()	Debe lanzar excepción cuando cliente no existe	Stephanie	Pasó
CP-AS09	createAccount()	Debe validar saldo inicial mayor a 0	Aracely	Pasó
CP-AS10	createAccount()	Debe rechazar saldo inicial negativo	Stephanie	Pasó
CP-AS11	deleteAccount()	Debe eliminar cuenta existente	Stephanie	Pasó
CP-AS12	deleteAccount()	Debe lanzar excepción si cuenta no existe	Stephanie	Pasó
TransactionServiceTest				
CP-AS13	deposit()	Debe aumentar saldo correctamente	Aracely	Pasó
CP-AS14	deposit()	Debe lanzar excepción si cuenta no existe	Stephanie	Pasó
CP-AS15	deposit()	Debe rechazar depósito negativo	Aracely	Pasó
CP-AS16	deposit()	Debe rechazar depósito cero	Stephanie	Pasó
CP-AS17	withdraw() [cuenta de ahorro]	Debe permitir retiro con saldo suficiente	Aracely	Pasó
CP-AS18	withdraw() [cuenta de ahorro]	Debe rechazar retiro con saldo insuficiente	Aracely	Pasó
CP-AS19	withdraw() [cuenta de ahorro]	Debe rechazar retiro que deje saldo negativo	Stephanie	Pasó
CP-AS20	withdraw() [cuenta de corriente]	Debe permitir sobregiro dentro del límite (-500)	Aracely	Pasó
CP-AS21	withdraw() [cuenta de corriente]	Debe rechazar sobregiro fuera del límite (-500)	Stephanie	Pasó
CP-AS22	withdraw() [cuenta de corriente]	Debe tener éxito en caso de límite exacto	Stephanie	Pasó
AccountControllerTest				
CP-AS23	GET /accounts	Retorna 200 OK con lista de cuentas	Aracely	Pasó
CP-AS24	GET /accounts/{id}	Retorna 200 OK con cuenta	Aracely	Pasó
CP-AS25	GET /accounts/customer/{customerId}	Retorna 200 OK con cuentas del cliente	Stephanie	Pasó
CP-AS26	POST /accounts	Retorna 201 Created con cuenta creada	Aracely	Pasó

CP-AS27	PUT /accounts/{accountId}/deposit	Retorna 200 OK con saldo actualizado	Aracely	Pasó
CP-AS28	PUT /accounts/{accountId}/withdraw	Retorna 200 OK con saldo actualizado	Stephanie	Pasó
CP-AS29	DELETE /accounts/{id}	Debe retornar 204 No Content	Stephanie	Pasó
GlobalExceptionHandlerTest				
CP-AS30	MethodArgumentNotValidException	Retorna 400 con errores de validación	Aracely	Pasó
CP-AS31	AccountNotFoundException	Retorna 404 con mensaje apropiado	Aracely	Pasó
CP-AS32	CustomerNotFoundException	Retorna 404 con mensaje apropiado	Stephanie	Pasó
CP-AS33	InsufficientBalanceException	Retorna 400 con mensaje apropiado	Stephanie	Pasó

TransactionControllerTest				
CP	Método	Descripción	Responsable	
CP-TS01	POST/transactions/deposit	Debe procesar depósito exitosamente 200 OK	Aracely	Pasó
CP-TS02	POST/transactions/deposit	Debe manejar error de validación 400 Bad Request	Aracely	Pasó
CP-TS03	POST/transactions/withdraw	Debe procesar retiro exitosamente 200 OK	Aracely	Pasó
CP-TS04	POST/transactions/withdraw	Debe manejar error de validación 400 Bad Request	Aracely	Pasó
CP-TS05	POST/transactions/transfer	Debe procesar transferencia exitosamente 200 OK	Andrea	Pasó
CP-TS06	POST/transactions/transfer	Debe manejar error de validación 400 Bad Request	Andrea	Pasó
CP-TS07	GET /transactions/record	Debe listar transacciones exitosamente 200 OK	Andrea	Pasó
TransactionServiceImplTest				
CP-TS08	registerDeposit()	Debe registrar depósito exitosamente	Andrea	Pasó
CP-TS09	registerDeposit()	Debe manejar error del WebClient correctamente	Andrea	Pasó
CP-TS10	registerWithdrawal()	Debe registrar retiro exitosamente	Andrea	Pasó
CP-TS11	registerWithdrawal()	Debe manejar error del WebClient correctamente	Andrea	Pasó
CP-TS12	registerTransfer()	Debe registrar transferencia exitosamente	Andrea	Pasó
CP-TS13	registerTransfer()	Debe manejar error del WebClient correctamente	Stephanie	Pasó
CP-TS14	listTransactions()	Debe listar transacciones exitosamente	Stephanie	Pasó
CP-TS15	handleAccountServiceError()	Debe manejar error 400 (saldo insuficiente)	Stephanie	Pasó
CP-TS16	handleAccountServiceError()	Debe manejar error 404 (cuenta no encontrada)	Stephanie	Pasó
CP-TS17	handleAccountServiceError()	Debe manejar otros errores de servicio	Stephanie	Pasó
TransactionContextTest				
CP-TS18	executeStrategy()	Debe ejecutar estrategia de depósito correctamente	Angi	Pasó
CP-TS19	executeStrategy()	Debe ejecutar estrategia de retiro correctamente	Angi	Pasó
CP-TS20	executeStrategy()	Debe ejecutar estrategia de transferencia correctamente	Angi	Pasó
CP-TS21	executeStrategy()	Debe manejar error cuando no encuentra estrategia	Angi	Pasó
CP-TS22	executeStrategy()	Debe manejar error cuando hay tipo incompatible	Angi	Pasó
DepositStrategyTest				
CP-TS23	execute()	Debe procesar depósito exitosamente	Angi	Pasó
CP-TS24	execute()	Debe manejar error del AccountWebClient correctamente	Angi	Pasó
CP-TS25	getSupportedType()	Debe retornar TransactionRequestDTO como tipo soportado	Angi	Pasó

CP-TS26	getStrategyName()	Debe retornar "DEPOSITO" con nombre de estrategia	Angi	Pasó
WithdrawalStrategyTest				
CP-TS27	execute()	Debe procesar depósito exitosamente	Aracely	Pasó
CP-TS28	execute()	Debe manejar error del AccountWebClient correctamente	Aracely	Pasó
CP-TS29	getSupportedType()	Debe retornar TransactionRequestDTO como tipo soportado	Aracely	Pasó
CP-TS30	getStrategyName()	Debe retornar "RETIRO" con nombre de estrategia	Aracely	Pasó
TransferStrategyTest				
CP-TS31	execute()	Debe procesar transferencia exitosamente	Stephanie	Pasó
CP-TS32	execute()	Debe manejar error del AccountWebClient correctamente	Stephanie	Pasó
CP-TS33	getSupportedType()	Debe retornar TransferRequestDTO como tipo soportado	Stephanie	Pasó
CP-TS34	getStrategyName()	Debe retornar "TRANSFERENCIA" con nombre de estrategia	Stephanie	Pasó
GlobalExceptionHandlerTest				
CP-TS35	AccountNotFoundException	Retorna 404 con mensaje apropiado	Andrea	Pasó
CP-TS36	InsufficientBalanceException	Retorna 400 con mensaje apropiado	Aracely	Pasó
CP-TS37	TransactionNotValidException	Retorna 400 con mensaje apropiado	Stephanie	Pasó
CP-TS38	handleGenericException	Retorna 500 con mensaje apropiado	Stephanie	Pasó
CP-TS39	handleValidationException	Retorna 400, con múltiples field errors	Stephanie	Pasó